Notes: How Grab Configured Their Data Layer to Handle Multi-Million Database Transactions a Day

February 24, 2026

1 Introduction

Grab processes millions of food and mart orders every day. The most critical and brittle part of the infrastructure is the database. Given the spiky nature of the traffic (e.g., massive surges during dinner times or special events), the database layer must ensure high availability and fast query times.

2 Query Types

To design the database layer, two main types of queries need to be handled:

- **Transactional Queries (OLTP):** Real-time operations like getting, creating, updating, or deleting an order.
- **Analytical Queries (OLAP):** Queries that fetch historical orders of a user or generate statistics grouped by attributes.

3 Design Goals

Grab's design goals for their database layer were:

1. **Stability:** Must handle high queries per second (QPS) and be highly available. A partial failure should only degrade performance, not cause a complete outage.
2. **Cost-Effectiveness:** Architecture should not inflate infrastructure costs unnecessarily for slow or simple queries.
3. **Consistency:** Strong consistency is required for transactional queries (e.g., immediate updates). Eventual consistency is acceptable for analytical queries.

4 Database Choices

Instead of a single database, Grab uses two specialized databases:

4.1 OLTP: DynamoDB

DynamoDB was chosen for the transactional database (single source of truth for ongoing orders) for the following reasons:

- **Managed Service:** Highly available and scalable out of the box.
- **Strong Consistency:** Provides consistent reads for primary keys.
- **Hot Partition Handling:** DynamoDB dynamically moves infrequently accessed data to a shared partition, dedicating full infrastructure capacity to "hot" partitions, seamlessly solving the hotkey problem.

Key Optimization: Partial Indexing in DynamoDB

To query a user's *ongoing* orders quickly without bloating the index:

- Instead of indexing the `user_id` directly, Grab created a special attribute called `user_id_gsi` and created a Global Secondary Index (GSI) on it.
- When an order is created (ongoing state), `user_id_gsi` is set to the `user_id`.
- Once the order is completed, `user_id_gsi` is set to `null`.
- Setting it to `null` removes the entry from the GSI. Thus, the index remains small (only containing ongoing orders) and lightning-fast to query.
- A TTL is set on the DynamoDB rows to delete orders after 3 months, keeping the operational table lean while historical data is passed to the analytics database.

4.2 OLAP: MySQL

While typical analytical loads use massive data warehouses, Grab's analytical requirements for orders weren't humongous enough to warrant an overkill data warehouse. They chose **MySQL** to serve near real-time analytical queries, structuring information to minimize cross-shard queries.

5 Data Synchronization & Architecture Pipeline

To keep the OLTP (DynamoDB) and OLAP (MySQL) databases in sync, Grab uses an event-driven ingestion pipeline:

- **Happy Path:** Order service updates DynamoDB → pushes events to Kafka → Consumers read from Kafka → updates MySQL.
- **High Availability/Fallbacks:** Kafka has an SLA of 99.95%. If Kafka is down, messages are pushed to **SQS**. If SQS is down, a **Dead Letter Queue (DLQ)** is configured.

Handling Out-of-Order Messages

Because asynchronous messages can arrive out of order:

- **Upserts:** Data is always inserted into MySQL using "upserts" to handle cases where an *update* event arrives before a *create* event.
- **Timestamp Checks:** To prevent a newer update from being overwritten by a delayed older update, consumers always verify the timestamp before applying an update.